WHAT IS A SYSTEM?

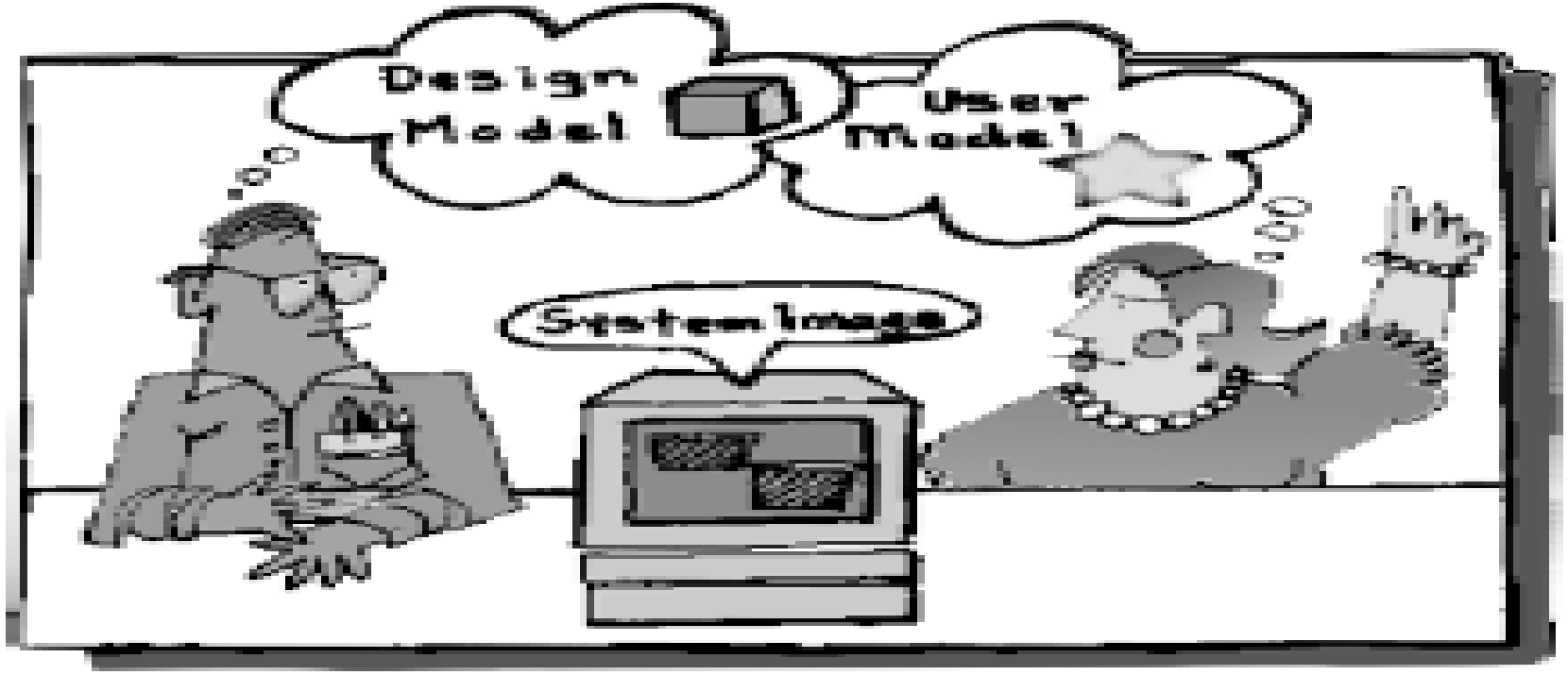
- A system has a specific set of objectives.
- Components of a system work together to achieve those objectives within a limited boundary.
- Components are given a specific set of tasks/ responsibilities.
- Components are related with each other and communicate with each other based on the type of the relationships.

QUALITY OF A SYSTEM

- Objectives must be well defined and specific
- Components must have well defined and specific responsibilities that they can perform independently.
- Communication among components should be as minimum as possible; only on need basis to transfer the output of one component to another component.

WHY WE MODEL?

- A model is a simplification of reality
- A model provides the blueprints of a system.

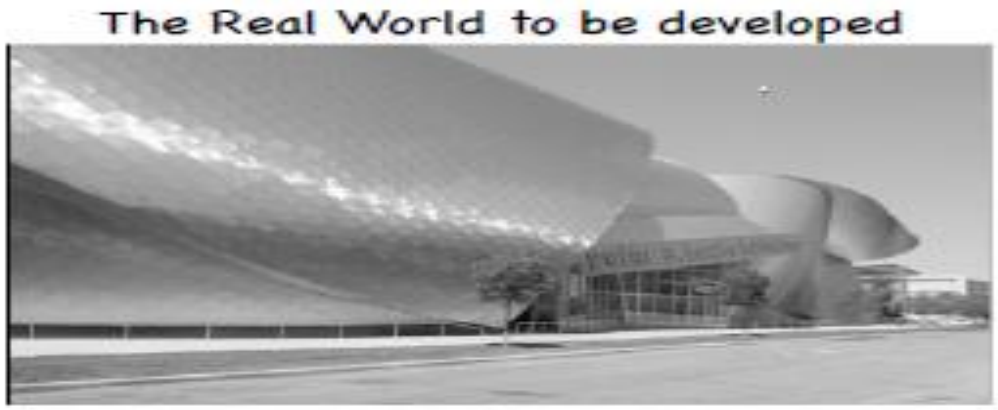


WHY WE MODEL?

- The larger and more complex the system, the more important modelling becomes, for one very simple reason:

  We build models of complex systems because we cannot fully mean of such a system in its entirety.

- There are limits to the human ability to understand complexity. Through modelling we narrow the problem we are studying by focusing on only **one aspect** at a time.





WHY WE MODEL?

- **Modeling** is the designing of software applications before coding.
- Modeling is an Essential Part of large software projects, and helpful to medium and even small projects as well.
- **Modelling** is a proven and well-accepted engineering technique.
- Unsuccessful software projects fail in their own unique ways, but all successful projects are alike in many ways.
- There are many elements that contribute to a successful software organization; one common thread is the use of **modelling**.
- We do model so that we can better understand the system we are developing.

BENEFITS OF MODEL

- Through modelling we achieve four aims:

1. Models help us to **visualize a system** as it is or as we want it to be
2. Models permit us to specify the **structure or behaviour** of a system
3. Models give us **a template that guides us in constructing** a system
4. Models **document the decisions** we have made

LIMITATION OF HUMAN ABILITY

- The larger and more complex the system, the more important modelling becomes, for one very simple reason:
  - *We build models of complex systems because we cannot comprehend such a system in its entirety.*
- There are limits to the human ability to understand complexity. Through modelling we narrow the problem we are studying by focusing on only one aspect at a time.

PRINCIPLES OF MODELLING – 1ST

- *The choice of what models to create has a profound influence on how a problem is attacked and how a solution is shaped.*

  -In other words, choose models intelligently.

  The right models will illuminate the wicked development problems.

PRINCIPLES OF MODELLING – 2ND

- *Every model may be expressed at different levels of precision.*

  - Sometimes a quick and simple executable model of the user interface is exactly what you need; at other times, one has to get down and dirty with the bits. In any case, the best kinds of models are those that let you choose your degree of detail, depending on who is viewing and why they need to view it.

PRINCIPLES OF MODELLING – 3RD

- *The best models are connected to reality.*

  - A Model connected to reality makes it more understandable and presentable. The more abstract a model is the more it needs to be closer to reality.

PRINCIPLES OF MODELLING – 4TH

- *No single model is sufficient.*

  - Every nontrivial system is best approached through a small set of nearly independent models.

WHAT IS UML?

- Unified Modeling Language
  - OMG Standard, Object Management Group
    Based on work from Booch, Rumbaugh, Jacobson
- UML is a modeling language to express and design documents, software:
  - Particularly useful for OO design
  - Not a process
  - Independent of implementation language
  - Version 0.9 released in 1996
  - Version 1.1 adopted by OMG in 1997
- Latest Version 2.5.1 (December 2017) - http://www.uml.org/

WHY USE UML

- Standardized graphical notation for
  - Specifying
  - Visualizing
  - Constructing and
  - Documenting software systems
- Language can be used from general initial design to very specific detailed design
- Increase understanding/communication of product to customers and developers
- Support for UML in many software packages today (e.g. Rational)

The UML is a Language for Visualizing

- Some things are best modeled textually; others graphically
- One developer can write a model in the UML, and another developer, or even another tool, can interpret that model unambiguously

The UML is a language for Specifying

- Specifying means building models that are precise, unambiguous, and complete
- In particular, the UML addresses the specification of all the important analysis, design, and implementation decisions that must be made in developing and deploying a software-intensive system.

The UML is a Language for Constructing

- UML is not a visual programming language, but its models can be directly connected to a variety of programming languages
- It is possible to map from a model in the UML to a programming language such as Java, C++, or VB, or even to tables in a RDBMS

The UML is a language for Documenting

- The UML addresses the documentation of a system's architecture and all of its details
- The UML also provides a language for expressing requirements and for tests
- Finally, the UML provides a language for modeling the activities of project planning and release management

BUILDING BLOCKS OF THE UML



- The vocabulary of the UML encompasses three kinds of building blocks:
  1. **Things** - abstractions that are first-class citizens in a model
     - Structural things
     - Behavioral things
     - Grouping things
     - Annotation things
  2. **Relationships** - tie these things together
  3. **Diagrams** - group interesting collections of things

Structural Things

- **Structural things** are the **nouns** of the UML model. These are the mostly static parts of a model, representing elements that are either conceptual or physical. There are eight kinds of structural things.

Behavioral Things

- Behavioral things are the dynamic parts of UML models. These are the **verbs** of a model, representing behavior over time and space. There are three kinds of behavioral things

Grouping Things

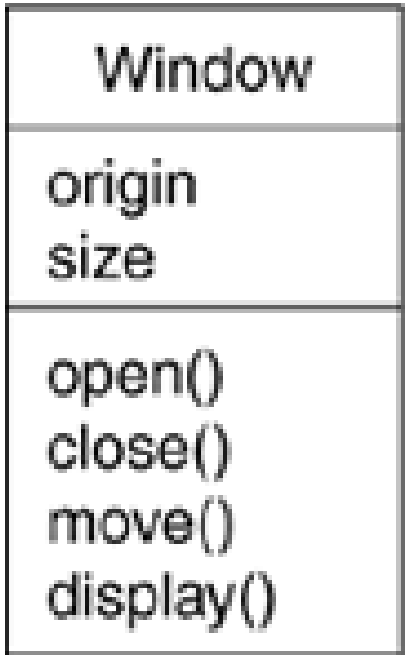
- Grouping things are the organizational parts of UML models. These are the boxes into which a model can be decomposed.
- There are also variations, such as frameworks, models, and subsystems (kinds of packages).

Annotational Things

- Annotational things are the explanatory parts of UML models. These are the comments you may apply to describe, illuminate, and remark about any element in a model.

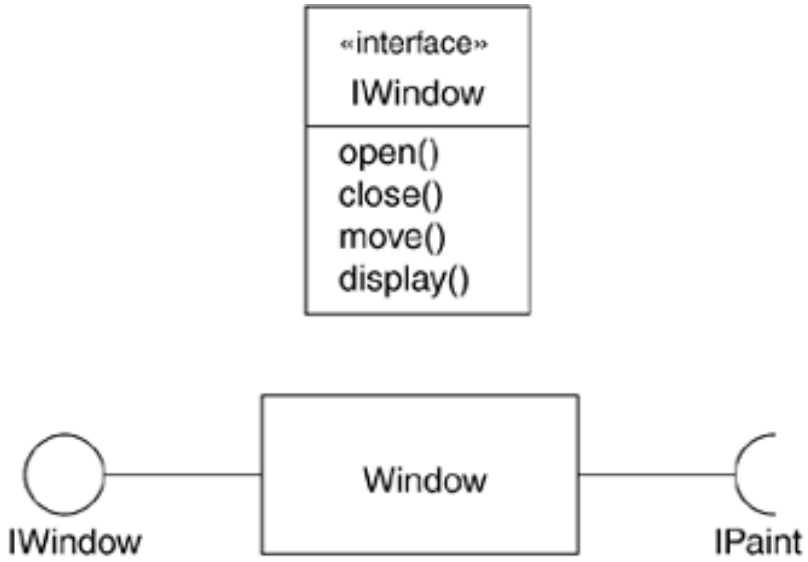
STRUCTURAL THING - CLASS

- **A class** is a description of set of objects that share the same attributes, operations, relationships, and semantics.



STRUCTURAL THING - INTERFACE

- **An interface** is a collection of operations that specify a service of a class or component. An interface therefore describes the externally visible behavior of that element
- <<Stereotype>> - Role



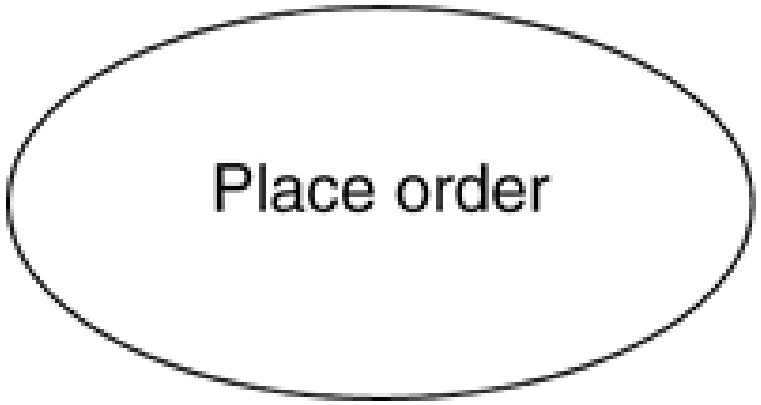
STRUCTURAL THING - COLLABORATION

- **A collaboration** defines an interaction and is a society of roles and other elements that work together to provide some cooperative behavior that's bigger than the sum of all the elements



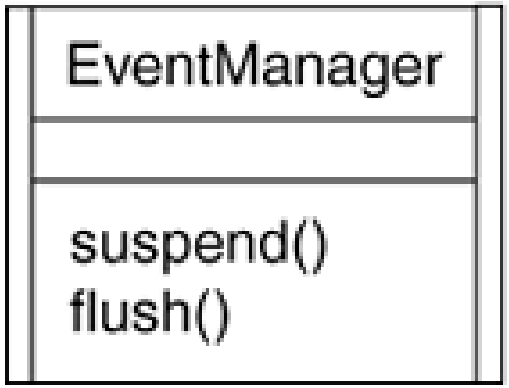
STRUCTURAL THING- USE CASE

- A **Use case** is a description of set of sequence of actions that a system performs that yields an observable result of value to a particular actor.



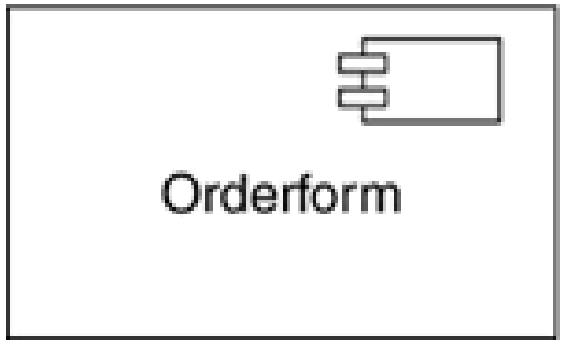
STRUCTURAL THING- ACTIVE CLASS

- **An active class** is a class whose objects own one or more processes or threads and therefore can initiate control activity. An active class is just like a class except that its objects represent elements whose behavior is concurrent with other elements.



STRUCTURAL THING- COMPONENT

- **A component** is a physical and replaceable part of a system that conforms to and provides the realization of a set of interfaces.



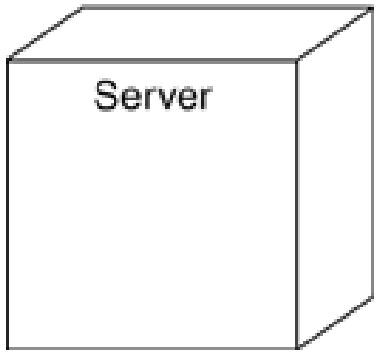
STRUCTURAL THING- ARTIFACT

- An **artifact** is a physical and replaceable part of a system that contains physical information ("bits").



STRUCTURAL THING- NODE

- **A node** is a physical element that exists at run time and represents a computational resource, generally having at least some memory and often, processing capability.



BEHAVIORAL THING- STATE MACHINE

- **A state machine** is a behavior that specifies the sequences of states an object or an interaction goes through during its lifetime in response to events.



BEHAVIORAL THING- ACTIVITY

- An **activity** is a behavior that specifies the sequence of steps a computational process performs.



BEHAVIORAL THING- INTERACTION

- **An interaction** is a behavior that comprises a set of messages exchanged among a set of objects within a particular context to accomplish a specific purpose.

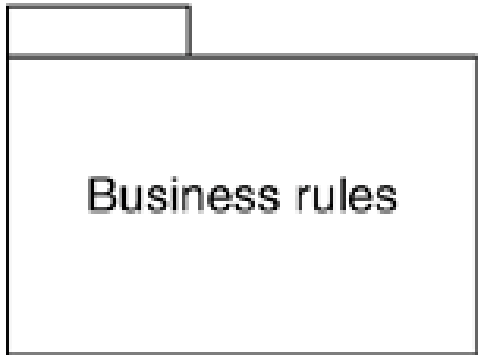


COMPARISON AMONG BEHAVIORAL THINGS

- **Comparison:**
  - In an interaction, the focus is on the set of objects that interact.
  - In a state machine, the focus is on the life cycle of one object at a time.
  - In an activity, the focus is on the flows among steps without regard to which object performs each step.
    - A step of an activity is called an **action**.

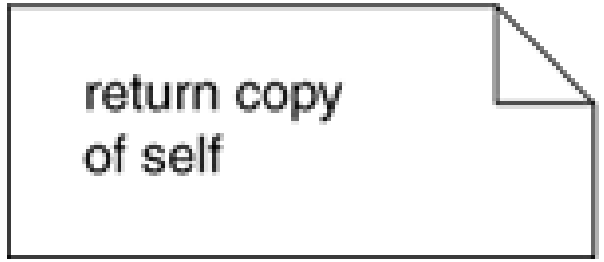
GROUPING THING- PACKAGE

- **A package** is a general-purpose mechanism for organizing elements into groups. Structural things, behavioral things, and even other grouping things can be placed in a package. Unlike components (which exists at runtime), a package is purely conceptual (meaning that it only exists at development time).



ANNOTATIONAL THINGS - NOTE

- There is one primary kind of annotational thing, called a **note**. A note is simply a symbol for rendering constraints and comments attached to an element or a collection of elements.



RELATIONSHIPS IN THE UML

- There are four kinds of relationships in the UML:
  1. Dependency
  2. Association
  3. Generalization
  4. Realization

Relationship - Dependency



- **A dependency** is a semantic relationship between two things in which a change to one (the independent thing) may effect the semantics of the other thing (the dependent thing).

Relationship - Association

- **An Association** is a structural relationship that describes a set of links. A link is a connection among objects.



Relationship - Generalization

- **A generalization** is a specialization/ generalization relationship in which objects of the specialized element (the child) are substitutable for objects of the generalized element (the parent). In this way, the child shares the structure and behavior of the parent.



Relationship - Realization

- **A realization** is a semantic relationship between classifiers, wherein one classifier specifies a contract that another classifier guarantees to carry out.

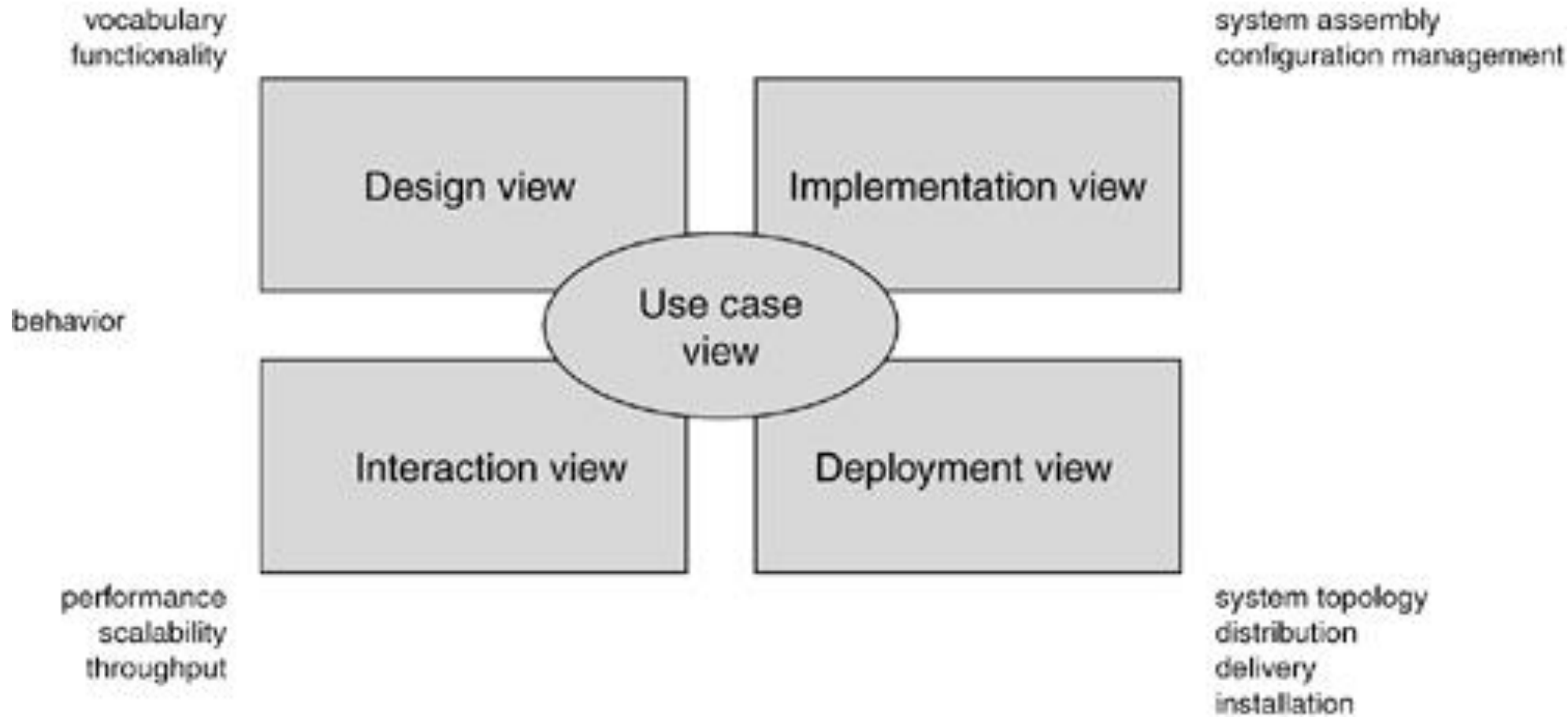


DIAGRAMS IN THE UML

A **diagram** is the graphical presentation of a set of elements, most often rendered as a connected graph of vertices (things) and arcs (relationships). The UML includes 14 such diagrams.

1. **Class diagram (Static Design or Process View)**
2. Object diagram (Static Design or Process View)
3. Component diagram (Static Design Implementation View)
4. Composite structure diagram (Hybrid)
5. **Use case diagram (Static Use Case View)**
6. **Sequence diagram (Dynamic Interaction View with Time Ordering)**
7. **Communication diagram (Dynamic Interaction View with structure)**
8. **State diagram (Dynamic Behavioral View of object)**
9. **Activity diagram (Dynamic Process View)**
10. Deployment diagram (Static Deployment View)
11. Artifact diagram (Static Implementation view)
12. Package diagram (Static View)
13. Timing diagram (Dynamic view with specific time)
14. Interaction overview diagram (Hybrid)

MODELING A SYSTEM'S ARCHITECTURE



UML ARCHITECTURE

- Visualizing, specifying, constructing and documenting a software intensive system demands that the system be viewed from a number of perspectives (e.g., ATM system development design in several diagram drawing perspective).
- Different stakeholders- end users, analysts, developers, system integrators, testers, technical writers and project managers- each bring different agenda to a project, and each looks at that system in different ways at different times over the project's life.

1. The **use case view** of a system encompasses the use cases that describe the behavior of the system as seen by its end users, analysts, and testers. With the UML, the static aspects of this view are captured in use case diagrams; the dynamic aspects of this view are captured in interaction diagrams, state diagrams, and activity diagrams.

UML ARCHITECTURE

2. The **design view** of a system encompasses the classes, interfaces, and collaborations that form the vocabulary of the problem and its solution. This view primarily supports the functional requirements of the system, meaning the services that the system should provide to its end users. With the UML, the static aspects of this view are captured in class diagrams and object diagrams; the dynamic aspects of this view are captured in interaction diagrams, state diagrams, and activity diagrams.

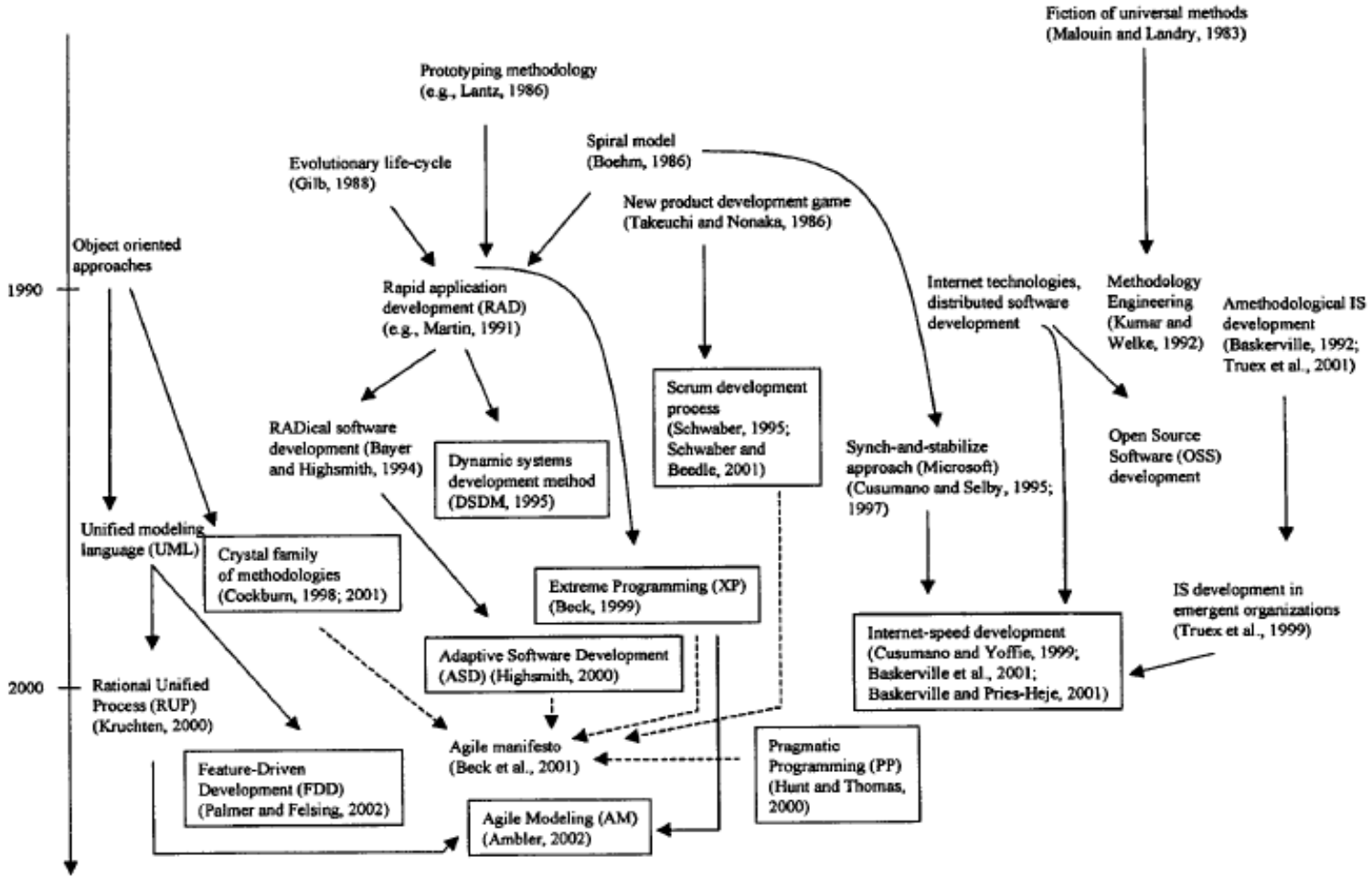
3. The **interaction view** of a system shows the flow of control among its various parts, including possible concurrency and synchronization mechanisms. This view primarily addresses the performance, scalability, and throughput of the system. With the UML, the static and dynamic aspects of this view are captured in the same kinds of diagrams as for the design view, but with a focus on the active classes that control the system and the messages that flow between them.

UML ARCHITECTURE

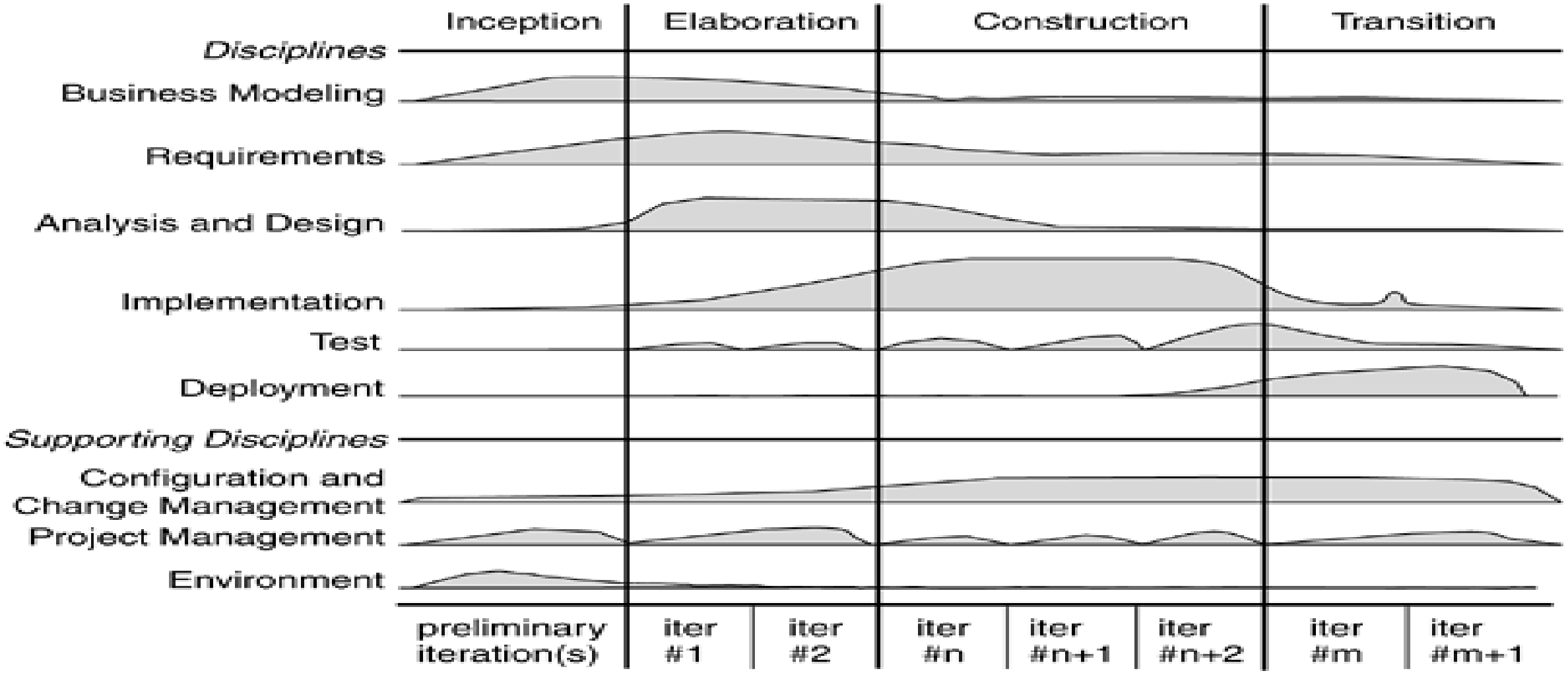
4. The **implementation view** of a system encompasses the artifacts that are used to assemble and release the physical system. This view primarily addresses the configuration management of the system's releases, made up of somewhat independent files that can be assembled in various ways to produce a running system. It is also concerned with the mapping from logical classes and components to physical artifacts. With the UML, the static aspects of this view are captured in artifact diagrams; the dynamic aspects of this view are captured in interaction diagrams, state diagrams, and activity diagrams.

5. The **deployment view** of a system encompasses the nodes that form the system's hardware topology on which the system executes. This view primarily addresses the distribution, delivery, and installation of the parts that make up the physical system. With the UML, the static aspects of this view are captured in deployment diagrams; the dynamic aspects of this view are captured in interaction diagrams, state diagrams, and activity diagrams.

EVOLUTION OF SOFTWARE DEVELOPMENT METHODOLOGIES



SOFTWARE DEVELOPMENT LIFE CYCLE – RATIONAL UNIFIED PROCESS (RUP)



SOFTWARE DEVELOPMENT LIFE CYCLE

- **Inception** is the first phase of the process, when the seed idea for the development is brought up to the point of being at least internally sufficiently well-founded to start the elaboration phase.
- **Elaboration** is the second phase of the process, when the product requirements and architecture are defined. In this phase, the requirements are articulated, prioritized, and **baselined**. A system's requirements may range from general vision statements to precise evaluation criteria, each specifying particular functional or nonfunctional behavior and each providing a basis for testing.
- **Construction** is the third phase of the process, when the software is brought from an executable architectural baseline to being ready to be transitioned to the user community. Here also, the system's requirements and especially its evaluation criteria are constantly reexamined against the business needs of the project, and resources are allocated as appropriate to actively attack risks to the project.
- **Transition** is the fourth phase of the process, when the software is delivered to the user community. Rarely does the software development process end here, for even during this phase, the system is continuously improved, bugs are eradicated, and features that didn't make an earlier release are added.
- An **iteration** is a distinct set of work tasks, with a **baselined** plan and evaluation criteria that results in an executable system that can be run, tested, and evaluated.